

File Permissions in Linux

Analyst: Mohammad
Environment: Linux Bash shell (Debian-based distribution)
Working directory: /home/researcher2/projects

Project Description

As a security professional working with a research team, part of my role is to ensure that users are authorized with appropriate permissions on the file system. In this project, I examined existing permissions on files and directories in the /home/researcher2/projects directory, identified any permissions that did not align with the organization's authorization policy, and used Linux commands to modify them accordingly. This work directly applies the principle of least privilege — ensuring users have only the access they actually need, and no more.

Check File and Directory Details

To check all permissions including hidden files, I used:

```
ls -la
```

The -l flag displays permissions, ownership, and file size in long format. The -a flag includes hidden files (those beginning with a period). Combining them as -la gives a complete picture of all permissions, including hidden entries. The hidden file .project_x.txt is visible here because of the -a flag. The group that owns all files in the projects directory is research_team.

Describe the Permissions String

Using project_k.txt as the example: -rw-rw-rw-

Position	Character	Meaning
1st	-	Regular file (not a directory)
2nd	r	User has read permission
3rd	w	User has write permission
4th	-	User does NOT have execute permission
5th	r	Group has read permission
6th	w	Group has write permission
7th	-	Group does NOT have execute permission
8th	r	Other has read permission
9th	w	Other has write permission — this is the violation
10th	-	Other does NOT have execute permission

Change File Permissions

project_k.txt had permissions -rw-rw-rw-, meaning other had write access — the only file with this violation. Command used:

```
chmod o-w project_k.txt
```

o targets other, - removes the permission, w specifies write. Result: -rw-rw-r--. Additionally, project_m.txt had group read access it should not have:

```
chmod g-r project_m.txt
```

Result: -rw----- — only researcher2 retains access.

Change File Permissions on a Hidden File

.project_x.txt is an archived hidden file. It should not be writable by anyone, but user and group should retain read access. Its permissions were -rw--w---- — user had write (should be removed), group had write only (write removed, read added). Command used:

```
chmod u-w,g-w,g+r .project_x.txt
```

Three operations in one command: u-w removes write from user, g-w removes write from group, g+r adds read to group. Multiple changes separated by commas with no spaces. Hidden files must be referenced with their leading period. Result: -r--r-----

Change Directory Permissions

Only researcher2 should access the drafts directory. Its permissions were drwx--x--- — the group had execute permission, meaning they could enter the directory. Command used:

```
chmod g-x drafts
```

Result: drwx----- — only researcher2 retains full access to drafts.

Summary

In this project, I used Linux Bash commands to examine and manage file and directory permissions for the researcher2 user's project files, applying the principle of least privilege throughout. Using ls -la, I identified four permission violations: other had write access to project_k.txt; group had read access to the restricted project_m.txt; the hidden archived file .project_x.txt had incorrect write permissions for both user and group; and the drafts directory allowed the group to execute (enter), which was not authorized. I used chmod to correct each violation, removing unauthorized access while preserving the permissions that legitimate users actually need.